WEEK 4

Week 4: Resource Management - Lab Guide

HPC & Cluster Computing

Objective: Transform your collection of nodes into a managed cluster using OpenHPC 4.1 methodology. **Goal**: Install the Slurm Workload Manager on the Master Node, inject it into your openEuler Compute Image, configure overlays, and submit your first parallel job.

---

1. Prerequisites & Environment Variables

Ensure your Master Node is running, and you have completed the Week 3 Lab. Because this lab session might be in a new terminal window, you **must load your environment variables** from Week 3 before proceeding.

Explanation: The OpenHPC installation tools rely on these bash variables. We will source the `cluster_env.sh` file we created in Week 3, and then export one additional variable specifically for Warewulf containers.

```
source cluster_env.sh
export CHROOT=$(sudo wwctl image show openeuler-24.03)
```

2. Install Munge (Authentication)

Slurm requires Munge to authenticate messages between nodes.

2.1 Install Munge on Master

Explanation: Installs Munge, an authentication service that Slurm uses to trust messages sent between the master and compute nodes.

```
sudo dnf install munge -y
```

2.2 Generate the Key

Explanation: Generates a secure cryptographic key (`munge.key`) and strictly restricts its read permissions, a security requirement for Munge to start.

```
sudo /usr/sbin/create-munge-key
sudo chown munge:munge /etc/munge/munge.key
sudo chmod 400 /etc/munge/munge.key
```

---

3. Install Slurm on Master Node

3.1 Install OpenHPC Server Packages

Explanation: Installs the OpenHPC-provided Slurm server packages that include the `slurmctld` daemon.

```
sudo dnf -y install ohpc-slurm-server
```

3.2 Configure `slurm.conf`

We use the OpenHPC templates and automatically replace the hostname with our `$sms_name` variable.

Explanation: Copies the default OpenHPC templates into the Slurm configuration directory and automatically substitutes the master hostname into the configuration using the `sed` utility.

```
sudo cp /etc/slurm/slurm.conf.ohpc /etc/slurm/slurm.conf
sudo cp /etc/slurm/cgroup.conf.example /etc/slurm/cgroup.
sudo sed -i "s/SlurmctldHost=\S\+/SlurmctldHost=${SMS_NAM
```

Then, open `/etc/slurm/slurm.conf` and add/edit the node and partition definitions at the bottom to match your cluster:

Explanation: Opens the configuration file in the nano editor so you can manually define the node resources.

```
sudo nano /etc/slurm/slurm.conf
```

```
# Add or modify these lines at the bottom:
NodeName=compute-01 NodeAddr=10.0.0.101 RealMemory=1000 S
```

4. Install Slurm into the Compute Image (Warewulf)

We execute commands inside our container image to install the OpenHPC slurm client and NTP tools.

Explanation: Uses `wwctl image exec` to run an installation script inside the compute node container image, downloading the Slurm client packages and enabling the Munge and Slurmd services for the next boot.

```
sudo wwctl image exec --build=false openeuler-24.03 -- /b
# Add Slurm client support meta-package
dnf -y install ohpc-slurm-client
# Enable services to start on boot
systemctl enable munge.service
systemctl enable slurmd.service
# Add Network Time Protocol (NTP) support
dnf -y install chrony
# Include modules user environment
dnf -y install lmod-ohpc
EOF
```

5. Import Configuration Overlays

5.1 Import Basic Configs

Explanation: Uses Warewulf Overlays to securely inject the host's subuid/subgid and Chrony configurations into the compute node dynamically, without baking them directly into the generic image.

```
# Import subuid and subgid
sudo wwctl overlay import --parents nodeconfig /etc/subui
sudo wwctl overlay import --parents nodeconfig /etc/subgi

# Identify head node as local NTP server
sudo wwctl overlay import --parents nodeconfig /opt/ohpc/
sudo wwctl profile set --yes nodes --tagadd ntpserver="${
```

5.2 Configure Slurm Overlays

Explanation: Copies the Slurm and Munge configuration files into the Warewulf nodeconfig overlay, ensuring they are identical on all compute nodes, and strictly applies the necessary file permissions required by Munge.

```
# Configure Slurm server in the overlay (using "configles
sudo wwctl overlay import --parents nodeconfig /opt/ohpc/

# Set the value of the slurmctld tag to the $SMS_IP
sudo wwctl profile set --yes nodes --tagadd slurmctld="${

# Configure munge
sudo wwctl overlay import --parents nodeconfig /etc/munge
sudo wwctl overlay chown --parents nodeconfig /etc/munge/munge.key
```

```
sudo wwctl overlay chown nodeconfig /etc/munge "$(id -u m
sudo wwctl overlay chmod nodeconfig /etc/munge 0700
```

6. Finalizing Provisioning

Rebuild the image to include the new packages and the overlays.

Explanation: Compiles the Warewulf overlays and container image to finalize all the changes we injected in the previous steps.

```
sudo wwctl image build openeuler-24.03
sudo wwctl overlay build
sudo wwctl configure --all
```

7. Start the Cluster

7.1 Start Slurm on Master

Explanation: Enables and starts the Slurm controller and Munge authentication daemon on the Master node.

```
sudo systemctl enable --now munge
sudo systemctl enable --now slurmctld
```

Reboot `compute-01` and `compute-02` from the console. They will download the newly image with Munge and Slurm installed.

7.3 Check Registration

Once the node is fully booted, check if it checked in with Slurm on the Master:

Explanation: Queries the Slurm controller to see the state of all managed compute partitions and nodes.

```
sinfo
```

---

8. Run Your First Job

Create a test job script named `job.sh`:

Explanation: Creates a batch script defining resource constraints (1 node) for the Slurm Workload Manager to execute.

```
#!/bin/bash
#SBATCH --job-name=first_job
#SBATCH --output=result.txt
#SBATCH --nodes=1

echo "Hello from \$(hostname)"
sleep 10
echo "Job Complete"
```

Explanation: Submits the batch script into the Slurm queue for execution on the compute node.

```
sbatch job.sh
```

---

9. Submission

Submit a PDF containing:

1. Screenshot of `sinfo` showing `compute-01` and `compute-02` in an `idle` state.
2. Screenshot of `squeue` showing your job running or pending.
3. The content of `result.txt` proving the job ran on `compute-01` successfully.